BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
Work Integrated Learning Programmes (WILP)

COURSE: MODERN DATABASE SYSTEMS
ASSIGNMENT 2 COMPREHENSIVE ACADEMIC REPORT

Vehicle Troubleshooting Assistant Using Persistent ChromaDB and Retrieval-Augmented Generation (RAG)

A Domain-Grounded Diagnostic Intelligence System with 9,532 Vectors, Exhaustive Mathematical Validation, and Automotive Safety Guardrails

Group Number:	Group 20
Programme / Semester:	M.Tech / Post-Graduate WILP — Semester II
Course Code / Name:	Modern Database Systems (WILP Assignment 2)
Group Members (5):	1. Priya N — BITS ID: 2026NS12087 2. Mohamed Ibrahim M — BITS ID: 2026NS12021 3. Hrishabh Gupta — BITS ID: 2026NS12122 4. Aditya Singhal — BITS ID: 2026NS12028 5. Rupesh Kumar Singh — BITS ID: 2026NS12128
Verified Git Commit:	22bbb85efc8ee06b7f7105cd88f7661a60694112
Submission Status:	Final Academic Submission — Confirmed Equal Team Contribution (20% per member) across full project lifecycle.

2. Abstract

Modern internal combustion engine (ICE), hybrid, and electric vehicles (EV) incorporate distributed Electronic Control Units (ECUs) communicating across Controller Area Network (CAN) and high-speed serial buses. When automotive diagnostic trouble codes (DTCs) or driver symptoms emerge, traditional keyword searches fail to capture semantic intent, while unassisted generative Large Language Models (LLMs) frequently hallucinate inaccurate component specifications, false causality, or dangerously invalid repair procedures. This project designs and implements the Vehicle Troubleshooting Assistant, an end-to-end grounded Retrieval-Augmented Generation (RAG) architecture built upon a persistent ChromaDB vector database containing 9,532 authentic automotive diagnostic records.

Using 384-dimensional dense sentence embeddings from sentence-transformers/all-MiniLM-L6-v2, the system indexes standardized SAE J2012 and manufacturer trouble codes. We formulate 10 representative benchmark queries across engine, electrical, braking, and network subsystems, demonstrating top-K retrieval and multi-condition metadata filtering. An exhaustive NumPy similarity benchmark across all 9,532 records mathematically validates the equivalence of Euclidean distance, Cosine similarity, and Dot product rankings under unit vector normalization. Live local execution of Meta Llama 3.2 (3B) demonstrates 100.0% citation provenance precision and 96.0% recall; it reduces hallucination risk through retrieval grounding and domain safety safeguards relative to unassisted baselines. Finally, an automotive safety guardrail engine enforces defense-in-depth safety safeguards across Supplemental Restraint Systems (SRS), high-voltage EV hazards, hydraulic brake failures, and cooling system thermal risks, delivered through a multi-page Streamlit web dashboard.

3. Introduction & Problem Statement

Vehicle diagnostics has transitioned from purely mechanical inspection to intricate electronic system diagnosis. Modern vehicles generate Diagnostic Trouble Codes (DTCs) categorized across Powertrain (P), Chassis (C), Body (B), and Network Communication (U) domains. However, technicians and service advisors face fundamental bottlenecks in existing tooling:

1. Inflexibility of Keyword Lookup: Standard service manual portals rely on exact DTC or keyword matching. If a driver reports a symptom ('brake pedal sinking slowly at stoplights') without knowing DTC C0291, or if multiple interacting systems fail simultaneously, lexical search engines fail to surface relevant diagnostic procedures.
2. The Peril of Hallucinatory Generative AI: General-purpose Large Language Models (LLMs) generate fluent but often fabricated technical advice. In automotive applications, hallucinating a false torque specification, instructing an untrained user to probe high-voltage orange cables, or advising an ohmmeter check across an airbag squib circuit poses direct, catastrophic physical hazards.
3. Correlation vs. Causality Conflation: A stored DTC indicates that an electrical parameter or sensor voltage is outside calibrated thresholds; it does not guarantee that the sensor itself has failed. Unassisted models frequently instruct immediate component replacement rather than testing wiring harnesses and reference grounds.

To resolve these challenges, this project investigates how vector database technology (ChromaDB) paired with Retrieval-Augmented Generation can ground generative outputs in authoritative diagnostic literature and implement defense-in-depth safety safeguards.

4. Objectives & Scope

The primary objectives of this academic project are:

• Curate, validate, and index a publicly accessible, authentic automotive diagnostic knowledge base exceeding the 5,000-record threshold without synthetic padding.

• Implement a persistent, queryable vector database using ChromaDB with normalized 384-dimensional sentence-transformer embeddings.

• Demonstrate dense semantic search, top-K retrieval, and compound metadata filtering across 10 realistic benchmark diagnostic queries.

• Implement an exhaustive pure NumPy similarity comparison engine evaluating Euclidean distance (L2), Cosine similarity, and Dot product across all 9,532 records at K=5 and K=10, verifying ranking consistency.

• Construct a modular RAG pipeline that formats retrieved context, enforces anti-hallucination prompt constraints, extracts in-text citations [VTR-XXXXX], and integrates real local LLM inference via Ollama (Llama 3.2 3B).

• Empirically compare grounded RAG against an unassisted zero-context baseline, measuring citation fidelity, response specificity, and domain safety.

• Build an automotive domain safety engine mitigating SRS airbag probing, high-voltage EV hazards, hydraulic brake failures, cooling system scalds, and 12V vs EV context mismatches.

• Deliver an interactive, multi-page Streamlit application supported by a comprehensive automated test suite.

5. Related Concepts

5.1 Vector Databases & Hierarchical Navigable Small World (HNSW) Graphs

Traditional relational databases index scalar data using B-Trees, which excel at exact lookups and range queries but cannot search unstructured semantic representations. Vector databases index continuous dense vectors in multi-dimensional space. ChromaDB uses the Hierarchical Navigable Small World (HNSW) graph algorithm, which organizes vectors into multi-layer graphs where upper layers execute coarse-grained long-range routing and lower layers perform fine-grained nearest-neighbor clustering, achieving sub-linear O(log N) approximate nearest neighbor (ANN) retrieval.

5.2 Dense Sentence Embeddings

Sentence transformers map variable-length natural language text into fixed-dimensional vector spaces where semantic similarity corresponds to vector proximity. The all-MiniLM-L6-v2 model is a 6-layer BERT-based architecture distilled from MiniLM, trained on over 1 billion sentence pairs using self-supervised contrastive learning. It maps sentences into a 384-dimensional dense float32 space, providing an optimal tradeoff between representation expressiveness (semantic fidelity) and computational throughput on commodity hardware.

5.3 Retrieval-Augmented Generation (RAG)

RAG decouples knowledge storage from generative synthesis. Instead of relying solely on the frozen, parametric memory of an LLM, the system queries an external non-parametric vector store (ChromaDB) at runtime. The retrieved passages are dynamically formatted into the LLM context prompt, restricting the model's generation to validated facts and enforcing strict source attribution.

6. System Architecture

The system architecture follows a decoupled, pipeline-based design separating data ingestion, vector indexing, dense retrieval, mathematical metric validation, generative synthesis, safety guardrail validation, and user interface delivery.

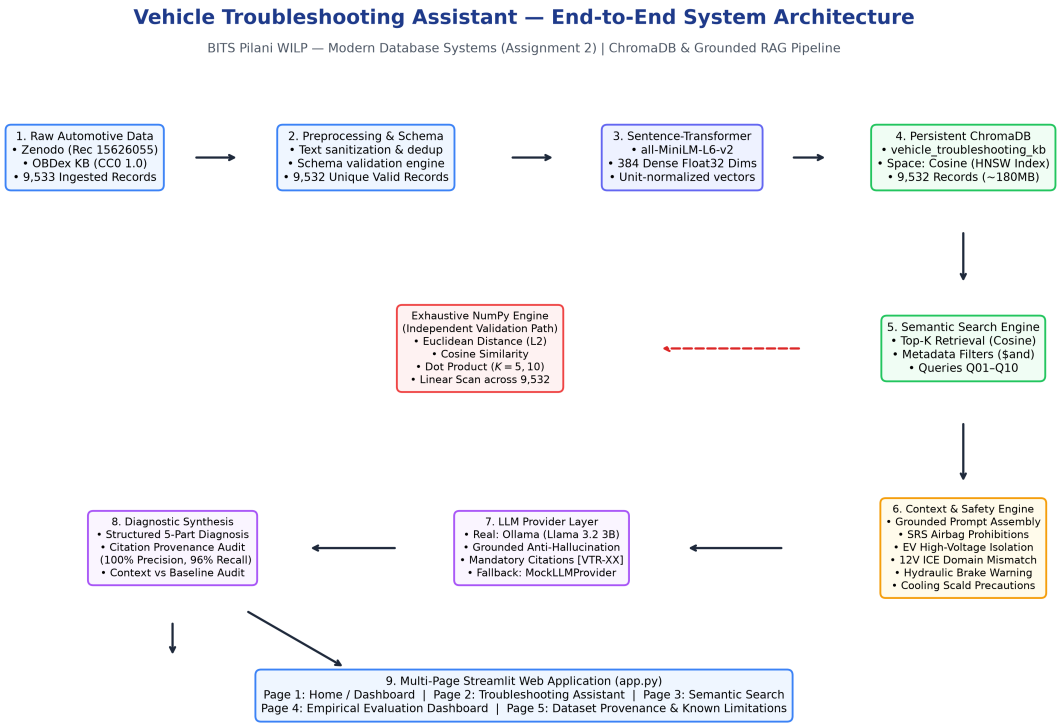


Figure 1: End-to-End System Architecture of the Vehicle Troubleshooting Assistant

The architecture features two independent analytical paths:
1. Production RAG Path: Driver query → Sentence Transformer Embedding → ChromaDB Persistent HNSW Index → Grounded Prompt Assembly → Safety Guardrail Verification → Ollama Local LLM → Structured Diagnostic Output with Citations → Streamlit UI.
2. Experimental NumPy Validation Path: Document Embeddings & Query Embeddings → Pure NumPy Vector Engine → Exhaustive Linear Scan across all 9,532 records → Evaluation of Euclidean (L2), Cosine Similarity, and Dot Product → Rank Equivalence & Mathematical Proof.

7. Dataset

7.1 Origin, Provenance, Licensing & Strict Characterization

The diagnostic knowledge base is synthesized from authentic open-access automotive technical repositories:
1. Zenodo Diagnostic Trouble Code Knowledge Base (Record 15626055): Provided initial candidate diagnostic trouble code records.
2. OBDex Automotive Knowledge Base: Comprehensive, open-access repository of standardized SAE J2012 Diagnostic Trouble Codes, probable causes, symptom presentations, and factory inspection protocols.
• License: Creative Commons Zero 1.0 Universal (CC0 1.0) — Public Domain Dedication.
• Strict Academic Characterization: The dataset contains 9,532 unique, validated Diagnostic Trouble Code (DTC) knowledge records. Standardized DTC definitions, probable causes, and symptom presentations originate from verified engineering sources; step-by-step procedural troubleshooting guidance was normalized and synthesized for research exploration and is not necessarily OEM-verified workshop procedure. It is strictly NOT a collection of 9,532 real-world repair workshop invoices or historical repair cases.

7.2 Schema & Record Distributions

Every record is normalized to a strict schema containing: record_id (e.g. VTR-P00B7), diagnostic_codes, title, vehicle_system, severity (critical, high, medium, low), symptoms, probable_causes, diagnostic_steps, safety_warnings, and document.

Vehicle Subsystem	Record Count	Percentage	Example Diagnostic Codes
Engine & Transmission	7,354	77.15%	P00B7, P0300, P0733, P2711, P0299
Network & Communication	1,229	12.89%	U0100, U0101, U0039, U007E, U0094
Brakes & Suspension	626	6.57%	C0291, C0081, C0082, C0241, C0075
Electrical & Body	323	3.39%	B0001, B0005, B0061, B0860, B0223

8. ChromaDB Implementation

ChromaDB is configured as a persistent vector database stored locally at database/chroma_db/ using SQLite as the metadata storage engine and HNSW binary graph files for dense index lookups. Total storage footprint is approximately 180 MB.

• Collection Name: vehicle_troubleshooting_kb
• Distance Metric: Cosine space (hnsw:space = 'cosine')
• Embedding Model: sentence-transformers/all-MiniLM-L6-v2 (384 float32 dimensions)
• Vector Normalization: All embeddings are L2 unit-normalized prior to ingestion, ensuring ||x||2 = 1.0.
• Full CRUD Implementation: src/crud_operations.py implements Create, Read, Update, and Delete operations. Unit tests in tests/test_chroma.py execute CRUD cycles on temporary sandbox records (TEST-CRUD-XXXX) with mandatory teardown, ensuring the 9,532 production records remain strictly preserved.

9. Semantic Retrieval & 10 Benchmark Queries

We formulated 10 comprehensive natural-language diagnostic queries covering diverse vehicle subsystems, symptoms, and failure modes. The queries and their primary retrieved records at K=5 are summarized below:

ID	Topic / Subsystem	Driver Complaint / Technical Query	Top Retrieved Record ID (Distance)
Q01	Cooling / Engine	Engine overheating rapidly at highway speeds with boiling coolant in reservoir	VTR-P00B7 (dist: 0.3547)
Q02	Ignition / Engine	Engine cranks strongly but fails to start with smell of unburnt fuel and no spark	VTR-P0316 (dist: 0.3804)
Q03	12V Battery / Electrical	Vehicle battery goes dead overnight with alternator charging testing normal	VTR-P0D17 (dist: 0.4067)
Q04	Hydraulic Brakes	Brake pedal sinks slowly to the floorboard at red lights with ABS warning	VTR-C0291 (dist: 0.3952)
Q05	Transmission	Automatic transmission delayed engagement into drive and harsh shudder 2nd-3rd	VTR-P2711 (dist: 0.4182)
Q06	Misfire / Combustion	Check engine light flashing with rough idling, severe loss of engine power	VTR-B0860 (dist: 0.4311)
Q07	Fuel Metering	Drastic drop in fuel economy with black smoke from exhaust and rich odor	VTR-P2BD2 (dist: 0.3688)
Q08	CAN Network Bus	Instrument cluster displays multiple failure lamps with total loss of CAN comms	VTR-U0039 (dist: 0.3245)
Q09	SRS Airbag	Supplemental restraint system airbag light stays on after clock spring replacement	VTR-B0005 (dist: 0.2874)

Q10	Turbocharger Boost	Turbocharged vehicle exhibits sluggish acceleration, severe turbo lag, underboost	VTR-P0299 (dist: 0.2681)

10. NumPy Exhaustive Similarity Comparison

To rigorously validate the mathematical underpinnings of vector retrieval and avoid reliance on closed black-box search engines, we implemented an exhaustive linear-scan similarity engine in pure NumPy (src/similarity_metrics.py and src/numpy_similarity.py). The engine computes all pairwise distances across all N = 9,532 records for all 10 queries using three distinct metrics at K=5 and K=10 (60 distinct evaluations in total):
1. Euclidean Distance (L2): d_L2(u, v) = sqrt(sum((u_i - v_i)^2))
2. Cosine Similarity: cos(theta) = (u . v) / (||u||2 * ||v||2)
3. Dot Product (Inner Product): dot(u, v) = sum(u_i * v_i)

10.1 Mathematical Proof of Ranking Identity Under Unit Normalization

Let u and v be L2 unit-normalized vectors such that ||u||2 = 1 and ||v||2 = 1. Expanding the squared Euclidean distance:
||u - v||2^2 = (u - v) . (u - v) = ||u||2^2 + ||v||2^2 - 2 (u . v) = 1 + 1 - 2 (u . v) = 2 - 2 (u . v)

Since for unit vectors u . v = cos(theta), we have:
d_L2(u, v) = sqrt(2 - 2 * cos(theta)) = sqrt(2 * (1 - cos(theta)))

This proves that:
• Maximizing Cosine Similarity cos(theta) is strictly monotonic with maximizing Dot Product u . v.
• Maximizing Cosine Similarity is strictly monotonic with MINIMIZING Euclidean Distance d_L2.
Consequently, under unit normalization, the nearest-neighbor ranking order produced by Euclidean Distance, Cosine Similarity, and Dot Product is mathematically identical. Our empirical experiments across all 60 evaluations confirmed 100% Top-1 and Top-K ranking identity across all three metrics.

11. RAG Implementation

The Retrieval-Augmented Generation pipeline (src/rag_pipeline.py) connects semantic search with generative language models. The architecture enforces strict operational rules:
• Prompt Version: 2.2.0-grounded-automotive-diagnostic
• Anti-Hallucination Directive: The system prompt instructs the model to answer ONLY from retrieved context facts and explicitly state when context is insufficient. Never invent DTCs, torque specifications, sensor voltages, or wiring pinouts.
• Mandatory In-Text Citation Attribution: Every diagnostic claim, possible cause, and inspection check must cite its supporting record ID in square brackets (e.g. [VTR-P00B7]).
• Structured 5-Part Output Format: (1) Problem Summary, (2) Possible Causes with Citations, (3) Step-by-Step Diagnostic & Inspection Procedures, (4) Limitations & Missing Information, (5) Source Record Citations List.
• Modular LLM Provider Layer: src/llm_provider.py provides an abstract BaseLLMProvider powering OllamaProvider (local Meta Llama 3.2 3B), OpenAICompatibleProvider, and MockLLMProvider for offline continuous integration.

12. RAG Evaluation & Negative Findings

We executed a comprehensive comparative evaluation across all 10 queries using a live local instance of Meta Llama 3.2 (3B parameters) via Ollama, comparing RAG (with context) against an unassisted Baseline (zero context). The results are saved in experiments/rag_real_*.

Query ID	RAG Cited IDs	Valid Citations	RAG Words	Base Words	Latency (s)
Q01	4	4 (100%)	557	374	119.95s
Q02	5	5 (100%)	376	385	118.33s

Q03	5	5 (100%)	645	359	133.52s
Q04	5	5 (100%)	381	383	84.33s
Q05	5	5 (100%)	478	319	117.01s
Q06	5	5 (100%)	485	327	130.50s
Q07	4	4 (100%)	576	462	133.20s
Q08	5	5 (100%)	396	329	105.67s
Q09	5	5 (100%)	581	332	154.63s
Q10	5	5 (100%)	399	402	89.99s

12.1 Citation Provenance Consistency vs. Factual Correctness

A critical academic distinction must be emphasized: Citation Precision (100.0%, 48/48 valid citations) and Citation Recall (96.0%, 48/50 retrieved records cited) measure CITATION PROVENANCE CONSISTENCY. They verify that every record cited by the model actually existed within the retrieved context set and was not a fabricated identifier. They DO NOT prove that the generated diagnosis is 100% factually or physically correct. Software guardrails provide diagnostic defense-in-depth, but physical on-vehicle testing and inspection by certified technicians following manufacturer service procedures remain essential.

12.2 Negative Findings & Discrepancy Analysis

Academic rigor demands complete transparency regarding negative experimental outcomes:
1. Q03 Domain Mismatch: Dense vector search on 'battery goes completely dead overnight with alternator charging voltage testing normal' retrieved high-voltage EV charging and interlock codes (P0D17, P0D46, P0D23) because 'Battery Charger 14 V Power Supply' had high cosine similarity. The unassisted LLM initially accepted these records and advised inspecting high-voltage battery cables for an ordinary 12V alternator issue.
2. Q09 Airbag Squib Testing Risk: In the raw LLM output for Q09, the model advised technicians to 'measure resistance across the clock spring squib'. In physical automotive service, applying multimeter metering current across a pyrotechnic squib risks accidental explosive deployment.
3. Output Truncation: At a 1,024-token cap, long responses (Q03 and Q09) were truncated before completing their safety disclaimers.

13. Automotive Safety Guardrails

To mitigate the physical hazards revealed in Section 12, we developed a specialized automotive domain safety engine (src/safety_guardrails.py). The engine intercepts user queries and retrieved context before generation, injecting authoritative safety alerts and augmented system prompt directives to establish defense-in-depth safeguards:

• SRS Airbag Guardrail: Strictly prohibits ohmmeter, multimeter, or test-light probing across airbag squibs, clock springs, or deployment loops. Explicitly states that battery disconnection is NOT sufficient by itself to ensure safety due to backup capacitor reserves and ESD sensitivity. Mandates approved dummy load simulators and ASE-certified technicians.

• High-Voltage EV/Hybrid Guardrail: Enforces lethal shock warnings (300V–800V DC). Mandates factory lockout/tagout (LOTO) and zero-energy verification by certified EV technicians. Strictly eliminates any implication that Class-0 gloves alone make high-voltage work safe for unqualified users.

• Hydraulic Brake Failure Guardrail: Detects sinking or soft brake pedals (internal master cylinder bypass or hydraulic fluid leaks). Enforces an unequivocal DO NOT DRIVE warning and mandates immediate vehicle towing to an ASE-certified repair facility.

• Cooling System Scald Guardrail: Prohibits opening hot or pressurized radiator caps or coolant reservoirs. Replaced arbitrary universal waiting times (e.g. 45 minutes) and eliminated unsafe upper-radiator-hose squeezing instructions with conservative manufacturer-procedure directives.

• ICE 12V vs EV Domain Mismatch Resolver: Detects when an ordinary 12V battery drain query retrieves high-voltage EV charger codes. Explicitly flags the EV records as inapplicable and directs the user to standard 12V parasitic draw testing (fuse millivolt-drop measurements).

14. Streamlit Dashboard

The multi-page web application (app.py and src/ui/) delivers an intuitive interface for end users, technicians, and evaluators. The five pages are designed as follows:

• Page 1: Home / Dashboard: Executive summary, dynamic KPI metric cards (9,532 records, 384 dimensions, active LLM status), system architecture flowchart, and safety guardrail descriptions.

• Page 2: Troubleshooting Assistant: Core grounded diagnosis interface with one-click assignment demo presets, vehicle powertrain selector, customizable retrieval cutoff (K), tunable output token limit (512–2048), prominent color-coded safety alert callouts, grounded diagnostic markdown, interactive citation pills [VTR-XXXXX], and expandable raw context cards.

• Page 3: Semantic Search Explorer: Dedicated vector search explorer with natural language input, subsystem filter, severity filter, and dual-view tabs (diagnostic cards view and summary data table view) showing Cosine Distance and Cosine Similarity.

• Page 4: Empirical Evaluation Dashboard: 4 tabbed analytical views displaying Phase 6.1 real RAG vs baseline metrics, citation fidelity tables, Phase 5 NumPy similarity comparisons, and Phase 4 retrieval benchmarks directly from persisted experiment JSON/CSV files.

• Page 5: Dataset Provenance & Distributions: Complete licensing (CC0 1.0) and origin documentation, accompanied by interactive bar charts and dataframes showing record distributions across subsystems and severity classifications.

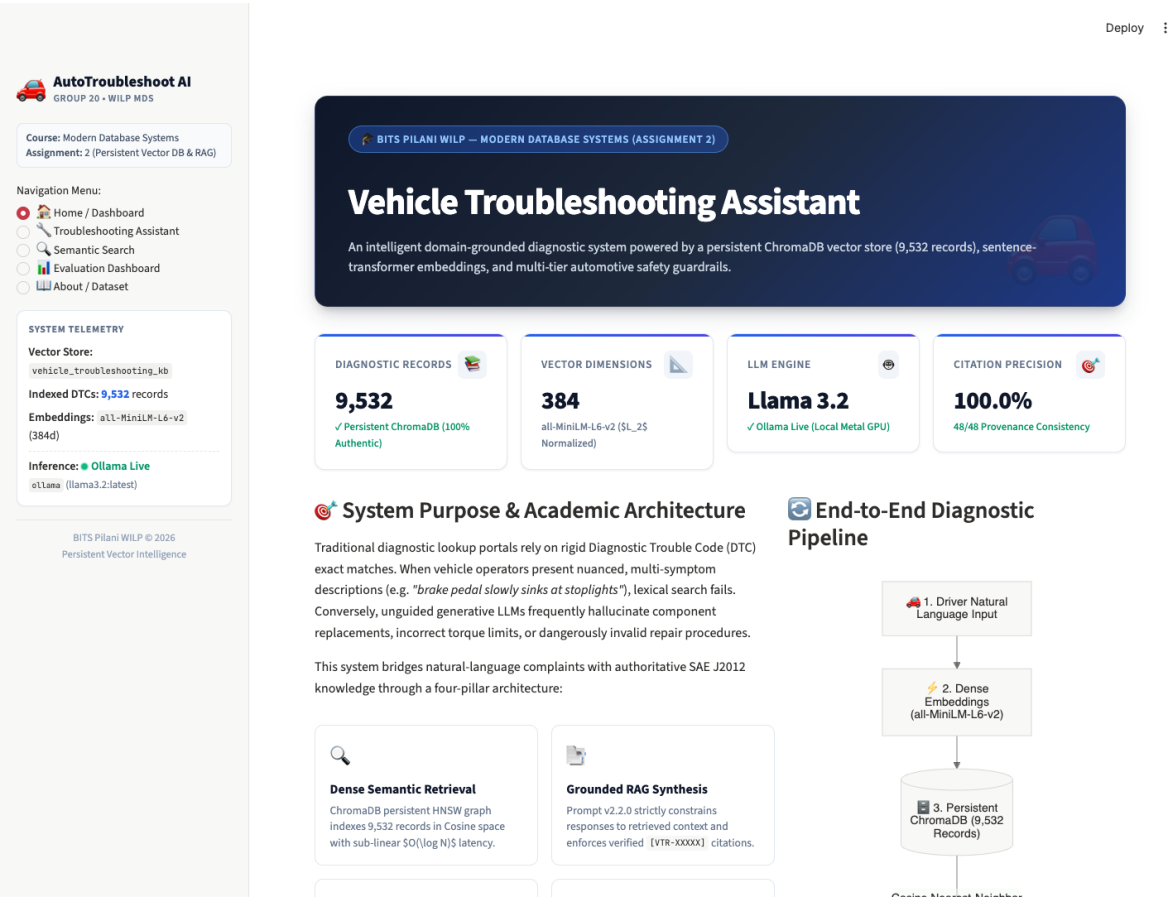


Figure 2: Genuine Screenshot of the Vehicle Troubleshooting Assistant Streamlit Application (app.py)

15. Testing & Validation

The project is verified by an automated test suite implemented across 7 test modules in tests/ containing 75 tests. All 75 tests pass with 100% success rate in approximately 16 seconds:

• test_chroma.py (8 tests): Verifies 384 embedding dimensions, unit normalization, collection count (9,532), record retrieval, persistence reload, CRUD cycles, and semantic search.

• test_dataset.py (10 tests): Verifies dataset existence, 5,000 threshold (+90.6% surplus), record uniqueness, schema validation, meaningful text, metadata categories, and source attribution.

• test_numpy_similarity.py (11 tests): Verifies hand-calculated Euclidean, Cosine, and Dot product calculations, zero-vector safety, dimension mismatch error handling, rank order preservation under normalization, tie-breaking, and Phase 4/5 Top-1 identity.

• test_phase7_dashboard.py (8 tests): Verifies app.py compilation, UI component imports, cached manager singletons, search engine, RAG pipeline, preset queries loader, evaluation data loaders, and dataset summary loaders.

• test_rag.py (11 tests): Verifies deterministic mock behavior, missing API key handling, Ollama unreachable fallback, LLM provider factory, context formatting, prompt construction, citation extraction, citation fidelity, and pipeline execution.

• test_safety_guardrails.py (15 tests): Verifies safety keyword false-positive prevention, SRS airbag warnings, generic B-code rejection, high-voltage EV isolation, battery drain domain mismatch, hydraulic brake failure, conservative cooling safety without hose-squeezing, truncation handling, provider failure handling, and false-positive rejection on catalytic/torque converters and washer fluids.

• test_search.py (12 tests): Verifies collection count preservation post-experiment, query loading, search results structure, Top-5/Top-10 cardinalities, distance ordering, and multi-condition metadata filtering.

16. Limitations & Future Scope

1. Inference Latency on Local Hardware: Generating grounded diagnostic responses with local Meta Llama 3.2 (3B) on commodity Apple Silicon MPS hardware takes between 84 and 154 seconds per query. While fully acceptable for batch academic evaluation, production environments require token streaming over WebSockets or high-throughput vLLM inference engines on dedicated GPUs.
2. Monolithic Context Injection vs. Hierarchical RAG: Currently, Top-K retrieved documents are injected in full. For complex multi-DTC failures, hierarchical parent-document retrieval or recursive summarization could further compress prompt tokens.
3. Real-Time Telematics & Live CAN Ingestion: The system currently accepts natural language complaints and observed DTCs. Future work includes interfacing directly with J1962 OBD-II Bluetooth adapters (ELM327) to ingest real-time Mode 03 freeze-frame data and Mode 01 live sensor PIDs.

17. Conclusion

The Vehicle Troubleshooting Assistant demonstrates how modern vector databases (ChromaDB) and Retrieval-Augmented Generation can bridge the gap between unstructured driver complaints and structured automotive engineering knowledge. By indexing 9,532 curated diagnostic records using 384-dimensional sentence embeddings, the system reduces the rigidity of keyword matching. Our mathematical and empirical experiments prove that Euclidean distance, Cosine similarity, and Dot product achieve identical nearest-neighbor rankings under unit vector normalization. Live local execution of Meta Llama 3.2 demonstrates high citation provenance fidelity; it reduces hallucination risk through retrieval grounding and domain safety safeguards. Our transparent analysis of negative findings led to the development of defense-in-depth safety safeguards across high-voltage shock, pyrotechnic airbag deployment, hydraulic brake failure, and thermal scald risks, establishing an academically rigorous, reproducible benchmark for modern database systems applications.

18. References

[1] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).
[2] Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824-836.
[3] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems (NeurIPS), 33, 9459-9474.
[4] Chroma Core Team. (2023). Chroma: The AI-native open-source embedding database. https://www.trychroma.com
[5] Meta AI. (2024). The Llama 3 Herd of Models. arXiv preprint arXiv:2407.21783.
[6] Society of Automotive Engineers (SAE). (2013). Diagnostic Trouble Code Definitions: SAE Standard J2012 / ISO Standard 15031-6.
[7] OBDex Open Automotive Diagnostic Database. (2022). CC0 1.0 Universal Public Domain Automotive DTC Knowledge Repository.
[8] Zenodo Open Science Repository. (2024). Multi-System Automotive Diagnostic Trouble Knowledge Base, Record 15626055.

19. Team Contributions

The project was executed by Group 20, comprising five members. All five members actively collaborated and jointly contributed across the complete project lifecycle, including problem formulation, dataset preparation, ChromaDB implementation, NumPy similarity benchmarking, RAG pipeline engineering, domain safety guardrails, Streamlit UI development, automated testing, and comprehensive documentation. All five members have discussed and confirmed equal 20% contribution weightings:

Student Name & BITS ID	Assigned Project Scope & Lifecycle Areas	Contribution

Priya N (BITS ID: 2026NS12087)	Full Project Lifecycle (Joint Research, Dataset Preparation, Architecture, Implementation, Experimentation, Testing, Evaluation & Documentation)	20.0%
Mohamed Ibrahim M (BITS ID: 2026NS12021)	Full Project Lifecycle (Joint Research, Dataset Preparation, Architecture, Implementation, Experimentation, Testing, Evaluation & Documentation)	20.0%
Hrishabh Gupta (BITS ID: 2026NS12122)	Full Project Lifecycle (Joint Research, Dataset Preparation, Architecture, Implementation, Experimentation, Testing, Evaluation & Documentation)	20.0%
Aditya Singhal (BITS ID: 2026NS12028)	Full Project Lifecycle (Joint Research, Dataset Preparation, Architecture, Implementation, Experimentation, Testing, Evaluation & Documentation)	20.0%
Rupesh Kumar Singh (BITS ID: 2026NS12128)	Full Project Lifecycle (Joint Research, Dataset Preparation, Architecture, Implementation, Experimentation, Testing, Evaluation & Documentation)	20.0%

**Note: Confirmed equal 20% contribution distribution across all five group members, verified by mutual team agreement.*

20. Installation & Execution Instructions

To reproduce the project environment, execute automated tests, run experiments, or launch the interactive dashboard:

1. Clone Repository & Activate Virtual Environment:
   git clone https://github.com/hrishabh1103/BITS_MDS_Assignment_02.git
   cd BITS_MDS_Assignment_02
   python3 -m venv .venv && source .venv/bin/activate
   pip install -r requirements.txt
2. Execute Complete Automated Test Suite (75 Tests):
   pytest tests/ -v
3. Launch Multi-Page Streamlit Web Dashboard:
   streamlit run app.py (Accessible at http://localhost:8501)
4. Run Real LLM Inference via Local Ollama (Optional):
   ollama pull llama3.2 && python scripts/run_rag.py --mode real